Custom Project

Name: Haoran Qin
Student ID: 36606936
Date: 13 May 2026

Project Pitch

My project is a broadcast messaging system with one server and multiple clients. Clients connect to the server and use it to send messages to other clients in the same channel. The system supports channel-based communication, message broadcasting, and receiving messages from other connected clients. In the HD Project version, I will first build a CLI/TUI client, then extend the system with RPC support and a web-based interface.

This project excites me because it demonstrates important networking and backend concepts, including client-server communication, message routing, channel management, concurrency, and real-time data flow. A broadcast system is also easy to understand from a user perspective, but it provides enough technical depth to show strong C++ programming and system design skills.

Core Features

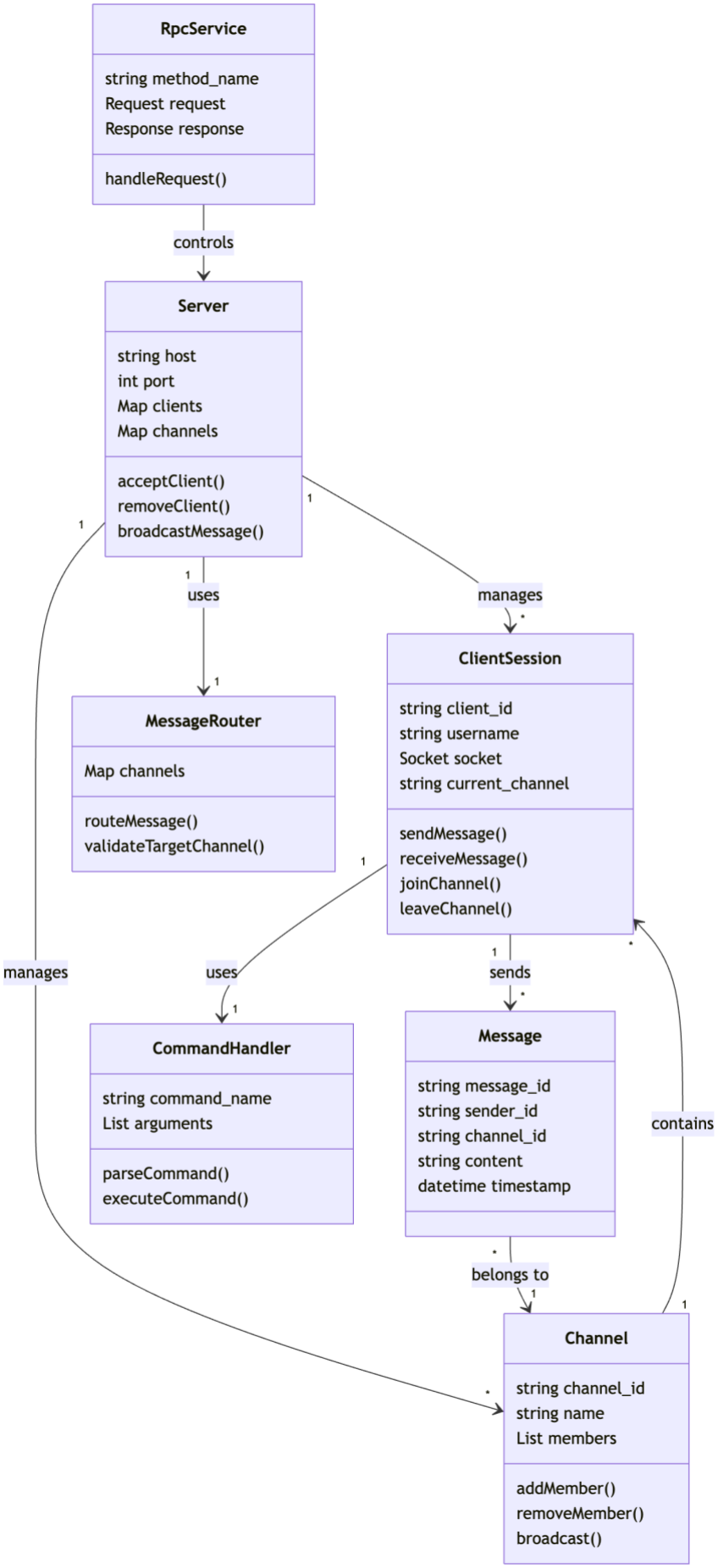
- Server: manages client connections, channels, and message broadcasting.
- Client: connects to the server, joins channels, sends messages, and receives broadcasts.
- Channel system: clients can send messages only to other clients in the same channel.
- Message routing: the server forwards messages to the correct channel members.
- CLI/TUI interface: users can interact with the system through a terminal-based interface.
- RPC extension: later versions will allow external programs or a web interface to communicate with the system.
- Web operation support: the HD Project will extend the system so users can operate it through a web UI.

Initial Data / Class Model

The system is organised around users, task lists, tasks, tags, comments, reminders, and audit logs. In C++ Drogon, these can be implemented as controller classes for APIs and model/data-access classes for PostgreSQL tables.

Object / Class	Main fields	Purpose
Server	host, port, clients, channels	Accepts client connections and manages broadcasting.

ClientSession	client_id, username, socket, current_channel	Represents one connected client.
Channel	channel_id, name, members	Groups clients so messages can be broadcast within the same channel.
Message	message_id, sender_id, channel_id, content, timestamp	Stores or represents one message sent through the system.
MessageRouter	channels, routing_rules	Decides which clients should receive each message.
CommandHandler	command_name, arguments	Processes CLI/TUI commands such as join, send, leave, and list.
RpcService	method_name, request, response	Supports future RPC-based control and web integration.


RpcService
string method_name
Request request
Response response
handleRequest()
controls
Server
string host
int port
Map clients
Map channels
acceptClient()
removeClient()
broadcastMessage()
uses
manages
MessageRouter
Map channels
routeMessage()
validateTargetChannel()
ClientSession
string client_id
string username
Socket socket
string current_channel
sendMessage()
receiveMessage()
joinChannel()
leaveChannel()
manages
uses
sends
CommandHandler
string command_name
List arguments
parseCommand()
executeCommand()
Message
string message_id
string sender_id
string channel_id
string content
datetime timestamp
contains
belongs to
Channel
string channel_id
string name
List members
addMember()
removeMember()
broadcast()

Using GenAI:

I use GenAI as a development assistant. I use it to generate boilerplate examples, suggest C++ header/source file structure, draft SQL migration templates, and review whether my database relationships and API naming are consistent. I will still decide the final architecture, write and test the code, debug compiler/database errors, and explain the implementation in my own words.